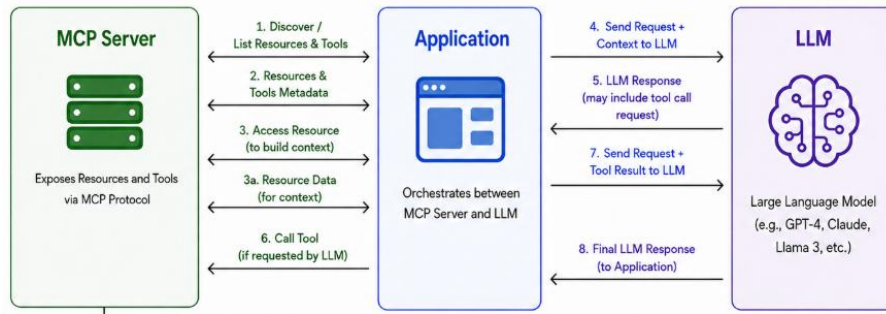


LLM Integration Flow

This image illustrates **LLM Interaction** in an MCP (Model Context Protocol) workflow, showing how a user query is processed through the MCP Client, MCP Server, and LLM.



The Flow:

1. Application / Agent discovers available Resources and Tools from the MCP Server.
2. MCP Server returns Resource metadata and Tool schemas to the Application / Agent.
3. Application / Agent accesses Resource(s) from the MCP Server to build context.
 - 3a. MCP Server returns Resource Data to the Application / Agent.
4. Application / Agent sends the Request + Context to the LLM.
5. LLM responds and may include a Tool Call Request.
6. Application / Agent calls the requested Tool on the MCP Server.
 - 6a. MCP Server executes the Tool and returns the Tool Result to the Application / Agent.
7. Application / Agent sends the Tool Result to the LLM.
8. LLM processes the Tool Result and returns the Final Response to the Application / Agent.

Using MCP with OpenAI API

The Schema of Tools used in MCP Server and OpenAI are not same and if the client app is using OpenAI API we have to convert the Schema from MCP to OpenAI format

MCP Tool Schema	OpenAI Tool Schema
<pre>{ "name": "get_news", "description": "Retrieve the latest news articles on a given topic.", "input_schema": { "type": "object",</pre>	<pre>{ "name": "get_news", "description": "Retrieve the latest news articles on a given topic.", "parameters": { "type": "function",</pre>

<pre> "properties": { "topic": { "type": "string", "description": "The topic to search news articles for." } }, "required": ["topic"] } </pre>	<pre> "properties": { "topic": { "type": "string", "description": "The topic to search news articles for." } }, "required": ["topic"], } </pre>
--	---

Server.py

```

from mcp.server.fastmcp import MCPServer

# Create an STDIO MCP server
mcp = MCPServer("Demo")

# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b + 10

@mcp.tool()
def subtract(a: int, b: int) -> int:
    """Subtract two numbers"""
    return a - b

@mcp.tool()
def multiply(a: float, b: float) -> float:
    """Multiply two numbers together and return the result."""
    return a * b

@mcp.tool()
def divide(a: float, b: float) -> float:

```

```

"""
Divide a by b and return the result.
Raises: ValueError: If b is zero
"""

if b == 0:
    raise ValueError("Cannot divide by zero")
return a / b

# Add a dynamic greeting resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"

# Main execution block - this is required to run the server
if __name__ == "__main__":
    mcp.run(transport="stdio") #local
    # mcp.run(transport="streamable-http") #remote

```

Using Math MCP Server with LLM

```

import asyncio
import json
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from openai import OpenAI

openai_client = OpenAI(
    api_key="your-openai-api-key",
    # base_url="https://api.openai.com/v1",
)

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

```

1. Fetch tools from MCP and convert to OpenAI format

```
mcp_tools = await session.list_tools()
openai_tools = [
    {
        "type": "function",
        "name": tool.name,
        "description": tool.description,
        "parameters": tool.inputSchema,
    }
    for tool in mcp_tools.tools
]
```

2. Send user message to OpenAI with tools available

```
user_message = "What is 42 plus 58?"
response = openai_client.responses.create(
    model="gpt-5",
    input=user_message,
    tools=openai_tools,
)
```

3. If LLM called a tool, execute it via MCP

```
for item in response.output:
    if item.type == "function_call":
        args = json.loads(item.arguments)
        print(f"LLM called '{item.name}' with args: {args}")

        mcp_result = await session.call_tool(item.name, args)
        tool_output = mcp_result.content[0].text
```

4. Send tool result back to OpenAI for final answer

```
final = openai_client.responses.create(
    model="gpt-5",
    previous_response_id=response.id,
    input=[{
```

```

        "type": "function_call_output",
        "call_id": item.call_id,
        "output": tool_output,
    }},
)
if (final.output_text):
    print("Final answer:", final.output_text)

```

```

asyncio.run(main())

```

Key points:

- `list_tools()` fetches all MCP tools dynamically — no hardcoding
- `previous_response_id` keeps conversation state server-side so you only send the tool result, not the full history
- This works for any tool the LLM picks, not just add

Multiple Tool Calls

```

import asyncio
import json
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from openai import OpenAI

openai_client = OpenAI(
    api_key="sk-proj-
kTSETIN9PAa5lcf72orjNRs3eQy1UqHpQNoXA_StlrgZlfU2J04GY29ULrhFJINvB_LjUMJHPT3BlbkFJLv4iUaqZJr
mmEWqePCNlgqPSeDyEo8orsZI1cl3YunkHxvXylclwqQwaGY1I487iSPjBKFDIQA",
    # base_url="https://api.openai.com/v1",
)

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])

```

```

async with stdio_client(server_params) as (read, write):

    async with ClientSession(read, write) as session:

        await session.initialize()

        # 1. Fetch tools from MCP and convert to OpenAI format

        mcp_tools = await session.list_tools()

        openai_tools = [

            {

                "type": "function",

                "name": tool.name,

                "description": tool.description,

                "parameters": tool.input_schema,

            }

            for tool in mcp_tools.tools

        ]

        # 2. Send user message to OpenAI with tools available

        user_message = "What is 42 plus 58 and 1 plus 3?"

        response = openai_client.responses.create(

            model="gpt-5.6-luna",

            input=user_message,

            tools=openai_tools,

        )

        # 3. If LLM called a tool, execute it via MCP

        inputs = []

        for item in response.output:

            if item.type == "function_call":

```

```
args = json.loads(item.arguments)

print(f"LLM called '{item.name}' with args: {args}")

mcp_result = await session.call_tool(item.name, args)

tool_output = mcp_result.content[0].text

inputs.append({

    "type": "function_call_output",

    "call_id": item.call_id,

    "output": tool_output,

})
```

4. Send tool result back to OpenAI for final answer

```
final = openai_client.responses.create(

    model="gpt-5.6-luna",

    previous_response_id=response.id,

    input=inputs

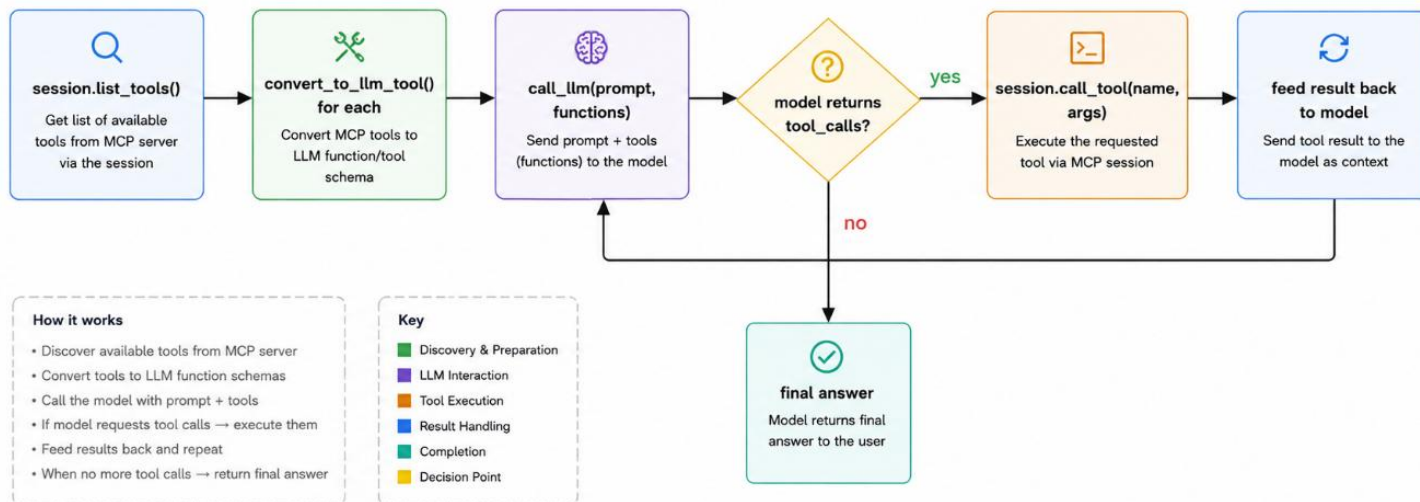
)

if (final.output_text):

    print("Final answer:", final.output_text)
```

```
asyncio.run(main())
```

Multiple Turns with Multiple Tool Calls



```

import asyncio
import json
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from openai import OpenAI

openai_client = OpenAI(
    api_key="sk-proj-
kTSETIN9PAa5lcf72orjNRs3eQy1UqHpQNoXA_StlrgZlfU2J04GY29ULrhFJINvB_LjUMJHPT3BlbkfJLv4iUaqZJr
mmEWqePCNlgqPSeDyEo8orsZI1cl3YunkHxvXylclwqQwaGY1I487iSPjBKFDIQA",
    # base_url="https://api.openai.com/v1",
)

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # 1. Fetch tools from MCP and convert to OpenAI format
            mcp_tools = await session.list_tools()
            openai_tools = [
                {

```



```

        "type": "function",
        "name": tool.name,
        "description": tool.description,
        "parameters": tool.input_schema,
    }
    for tool in mcp_tools.tools
]

# 2. Send user message to OpenAI with tools available
previous_response_id = None

inputs = [{"role": "user", "content": "What is 2 plus 8 and 1 plus 3 and then multiply the results
together."}]

while True:
    response = openai_client.responses.create(
        model="gpt-5.6-luna",
        input=inputs,
        tools=openai_tools,
        previous_response_id=previous_response_id,
    )
    if (response.output_text):
        print("Final answer:", response.output_text)
        break

    previous_response_id = response.id
    for item in response.output:
        if item.type == "function_call":
            args = json.loads(item.arguments)
            print(f"LLM called '{item.name}' with args: {args}")
            mcp_result = await session.call_tool(item.name, args)
            tool_output = mcp_result.content[0].text
            inputs.append({
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": tool_output,
            })

```

```
asyncio.run(main())
```

Institute Example with Multiple Tool Calls

SCENARIOS:

1. Find student Arjun Mehta and enroll him in batch 2. (2 calls: search_students → enroll_student)
2. Register a new student Ravi Kumar, email ravi@gmail.com, phone 9111111111. Then enroll him in batch 3." (2 calls: add_student → enroll_student),
3. "Show me who is in batch 1 and also list all students in the Python Full Stack course. (# 2 calls: get_batch_students + get_students_by_course)